

# LENGUAJES FORMALES

# ANÁLISIS SINTÁCTICO DESCENDENTE

Ing. Adolfo Andrés Castro Sánchez, MSc  
2026 - I

Área Ciencias Computacionales  
Escuela de Ciencias Aplicadas e Ingeniería

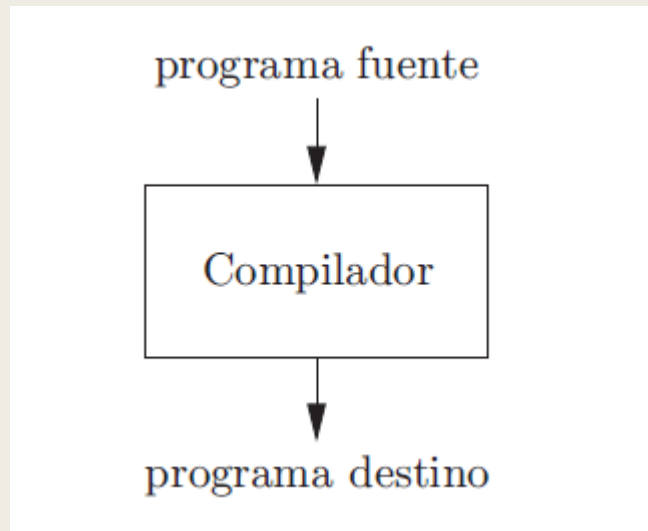
**UNIVERSIDAD**  
**EAFIT**

A thick black L-shaped frame is positioned on the left and bottom edges of the slide, framing the central text.

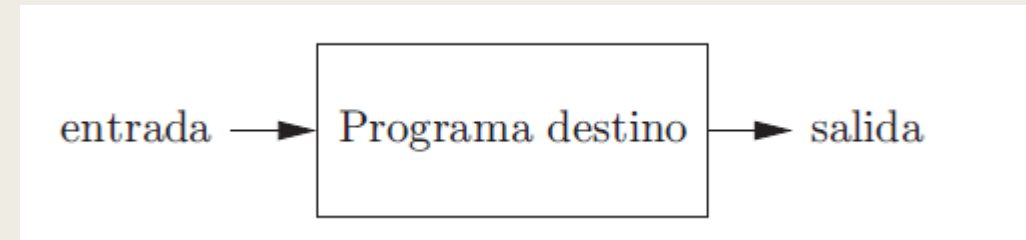
# **COMPILADORES**

Conceptos Básicos

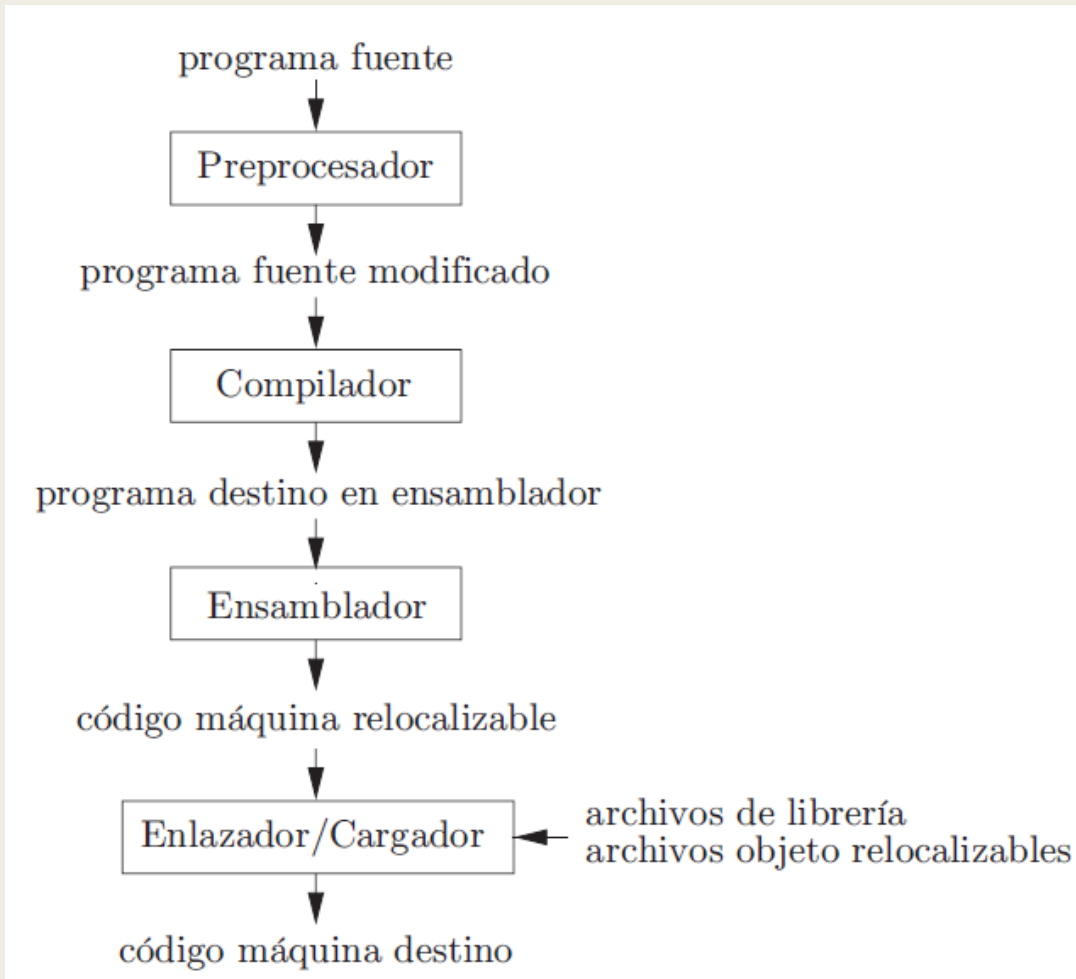
# Conceptos básicos



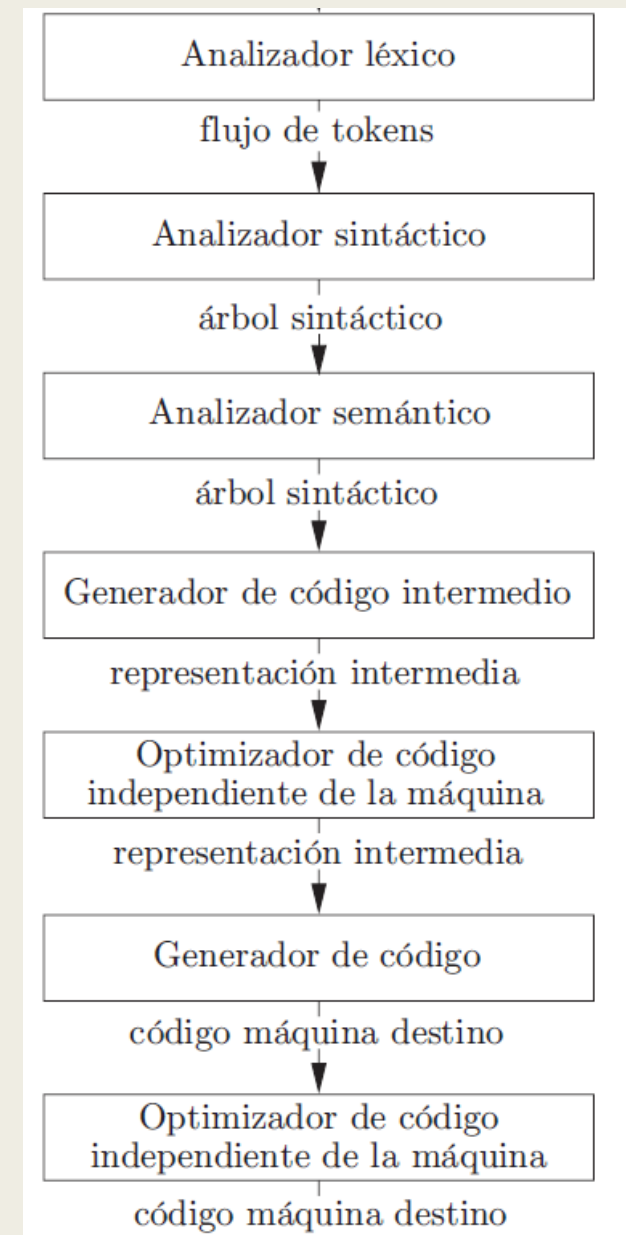
**COMPILADOR**



**PROGRAMA DESTINO**



## SISTEMA DE PROCESAMIENTO DEL LENGUAJE



## FASES DE UN COMPILADOR

# Traducción de instrucciones

`posicion = inicial + velocidad * 60`

Analizador léxico

`<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>`

Analizador sintáctico

```
graph TD
    A["<id, 1>"] --- B["="]
    C["<id, 2>"] --- B
    B --- D["+"]
    D --- E["<id, 3>"]
    D --- F["*"]
    F --- G["60"]
```

Analizador semántico

```
graph TD
    A["<id, 1>"] --- B["="]
    C["<id, 2>"] --- B
    B --- D["+"]
    D --- E["<id, 3>"]
    D --- F["*"]
    F --- G["inttofloat"]
    G --- H["60"]
```

Generador de código intermedio

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Optimizador de código

```
t1 = id3 * 60.0
id1 = id2 + t1
```

# Resultado final

Generador de código



```
LDF  R2, id3
MULF R2, R2, #60.0
LDF  R1, id2
ADDF R1, R1, R2
STF  id1, R1
```

Hablemos de proyecto Final



# **ANALIZADOR LÉXICO**





# Definición

A la primera fase de un compilador se le llama análisis de léxico o escaneo. El analizador de léxico lee el flujo de caracteres que componen el programa fuente y los agrupa en secuencias significativas, conocidas como lexemas.

Para cada lexema, el analizador léxico produce como salida un token de la forma:

*< nombre – token, valor – atributo >*

Ejemplo, suponga:

$$\textit{posicion} = \textit{inicial} + \textit{velocidad} * 60$$

1. ***posicion*** es un lexema que se asigna a un token  $\langle \textit{id}, 1 \rangle$ , en donde *id* es un *símbolo abstracto* y 1 apunta a la *entrada de memoria*.
2. El símbolo de asignación " = " es un lexema que se asigna al token  $\langle = \rangle$ . Este token *no necesita* un valor-atributo
3. ***inicial*** es un lexema que se asigna al token  $\langle \textit{id}, 2 \rangle$
4. + es un lexema que se asigna al token  $\langle + \rangle$ .
5. ***velocidad*** es un lexema que se asigna al token  $\langle \textit{id}, 3 \rangle$
6. \* es un lexema que se asigna al token  $\langle * \rangle$
7. 60 es un lexema que se asigna al token  $\langle 60 \rangle$

# Analizador Léxico

posicion = inicial + velocidad \* 60

Analizador léxico

$\langle \text{id}, 1 \rangle$   $\langle = \rangle$   $\langle \text{id}, 2 \rangle$   $\langle + \rangle$   $\langle \text{id}, 3 \rangle$   $\langle * \rangle$   $\langle 60 \rangle$

# Implementación del analizador léxico en Python

```
# Definir tipos de tokens
TOKEN_TYPES = [
    ('NUMBER', r'\d+(\.\d*)?'), # Números enteros o flotantes
    ('ID', r'[a-zA-Z_]\w*'), # Identificadores
    ('ASSIGN', r'='), # Operador de asignación
    ('PLUS', r'\+'), # Operador de suma
    ('MINUS', r'-'), # Operador de resta
    ('MULT', r'\*'), # Operador de multiplicación
    ('DIV', r'/'), # Operador de división
    ('LPAREN', r'\('), # Paréntesis izquierdo
    ('RPAREN', r'\)'), # Paréntesis derecho
    ('SEMICOLON', r';'), # Punto y coma
    ('WHITESPACE', r'\s+'), # Espacios en blanco
    ('UNKNOWN', r'.'), # Cualquier cosa no reconocida
]
```

```
class Lexer:
    def __init__(self, input_text):
        self.input_text = input_text
        self.tokens = []
        self.pos = 0

    def tokenize(self):
        while self.pos < len(self.input_text):
            match = None
            for token_type, pattern in TOKEN_TYPES:
                regex = re.compile(pattern)
                match = regex.match(self.input_text, self.pos)
                if match:
                    lexeme = match.group(0)
                    if token_type != 'WHITESPACE': #Ignorar espacio
                        self.tokens.append((token_type, lexeme))
                    self.pos = match.end(0)
                    break
            if not match:
                raise SyntaxError(f"Error de posición {self.pos}")
        return self.tokens
```

# Input/Output

```
# Ejemplo de uso
input_text = "x = 3 + 4.5 * (y - 2) ;"
lexer = Lexer(input_text)
tokens = lexer.tokenize()

for token in tokens:
    print(token)
```

```
('ID', 'x')
('ASSIGN', '=')
('NUMBER', '3')
('PLUS', '+')
('NUMBER', '4.5')
('MULT', '*')
('LPAREN', '(')
('ID', 'y')
('MINUS', '-')
('NUMBER', '2')
('RPAREN', ')')
('SEMICOLON', ';')
```

# SALIDA

```
('ID', 'x')
('ASSIGN', '=')
('NUMBER', '3')
('PLUS', '+')
('NUMBER', '4.5')
('MULT', '*')
('LPAREN', '(')
('ID', 'y')
('MINUS', '-')
('NUMBER', '2')
('RPAREN', ')')
('SEMICOLON', ';')
```

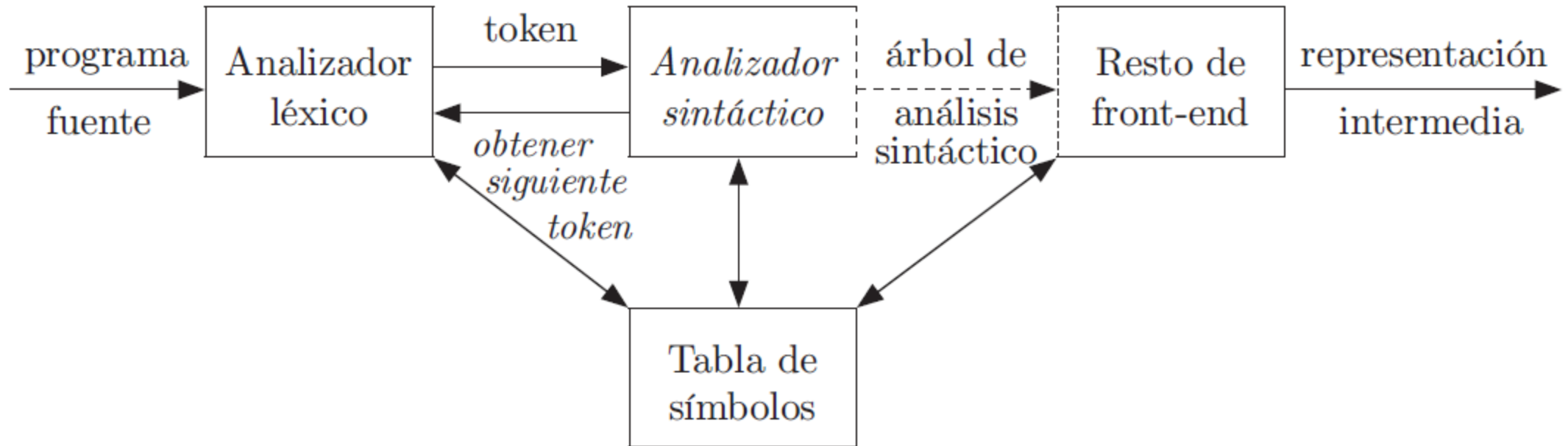


# **ANALIZADOR SINTÁCTICO**





# Función en el compilador



# Consideraciones

- Representación de gramáticas
- Manejo de errores
- Estrategia de recuperación de errores
- Gramáticas Libres de contexto y su relación con el parser
  - *Derivación*
  - *Árboles*
  - *Ambigüedad*
- Escritura de una gramática
  - *Comparación entre el análisis léxico y sintáctico*
  - *Eliminación de la ambigüedad*
  - *Eliminación de la recursividad por la izquierda*
  - *Factorización por la izquierda*
  - *Construcción para lenguaje que no son libres de contexto*

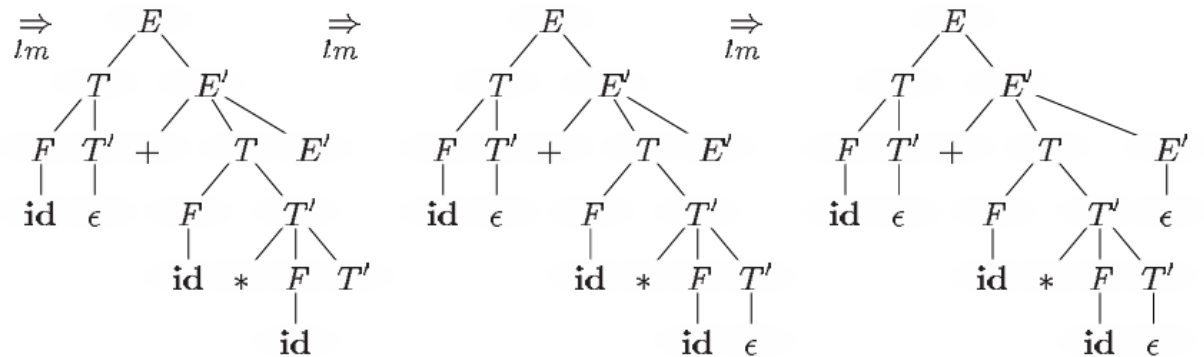
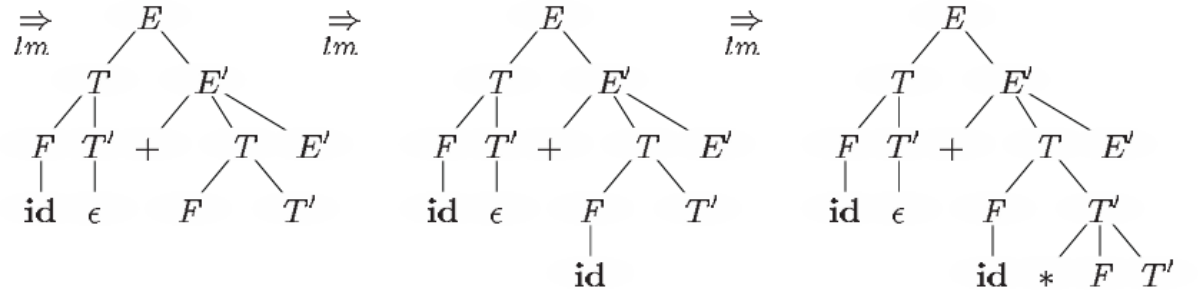
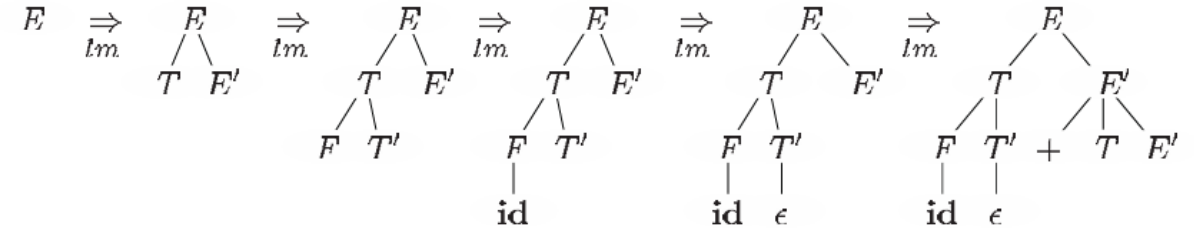
Esta información la pueden consultar en el libro de “Compiladores Principios y técnicas” de Alfred V Aho. El que tiene la portada del dragón. Capítulo 4 sección 4.1, 4.2 y 4.3

# Análisis sintáctico descendente

- El análisis sintáctico descendente puede verse como el problema de construir un *árbol de análisis sintáctico* para la cadena de entrada, *partiendo desde la raíz* y creando los nodos del árbol de análisis sintáctico en preorden. De manera equivalente, podemos considerar el análisis sintáctico descendente como la *búsqueda de una derivación por la izquierda* para una cadena de entrada.

$$\begin{array}{ll} E & \rightarrow T E' \\ E' & \rightarrow + T E' \mid \epsilon \\ T & \rightarrow F T' \\ T' & \rightarrow * F T' \mid \epsilon \\ F & \rightarrow ( E ) \mid \text{id} \end{array}$$

# Análisis Sintáctico: $id + id * id$

$$\begin{aligned}
 E &\rightarrow T E' \\
 E' &\rightarrow + T E' \mid \epsilon \\
 T &\rightarrow F T' \\
 T' &\rightarrow * F T' \mid \epsilon \\
 F &\rightarrow ( E ) \mid id
 \end{aligned}$$


# Análisis descendente Recursivo

- Un programa de análisis sintáctico de descenso recursivo consiste en un conjunto de procedimientos, uno para cada no terminal. La ejecución empieza con el procedimiento para el símbolo inicial, que se detiene y anuncia que tuvo éxito si el cuerpo de su procedimiento explora toda la cadena completa de entrada
- Rastreo hacía atrás: (Exploraciones repetidas sobre la entrada).
  - *Utilizado por algunos analizadores sintácticos*
  - *Mayor gasto computacional*
  - *Modificación del analizador para implementarlo*

# Ejemplo de pseudocódigo

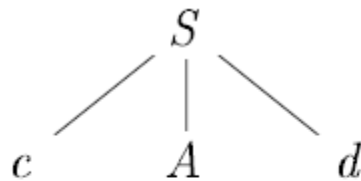
## Recursive-Descent Parsing

```
void A() {  
    Choose an A-production,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
    for (  $i = 1$  to  $k$  ) {  
        if (  $X_i$  is a nonterminal )  
            call procedure  $X_i()$ ;  
        else if (  $X_i$  equals the current input symbol  $a$  )  
            advance the input to the next symbol;  
        else /* an error has occurred */;  
    }  
}
```

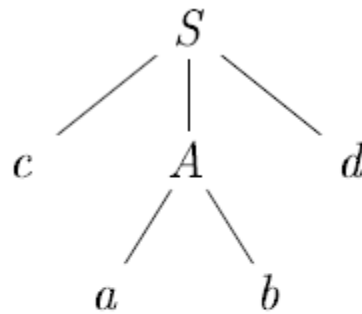
# Análisis para la cadena: $w = cad$

**Example 4.29:** Consider the grammar

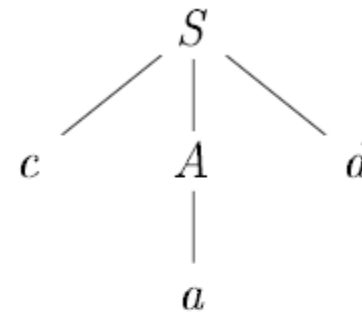
$$\begin{array}{lcl} S & \rightarrow & c A d \\ A & \rightarrow & a b \mid a \end{array}$$



(a)



(b)



(c)

Figure 4.14: Steps in a top-down parse

# Inconvenientes

- Una *gramática recursiva por la izquierda* puede hacer que un analizador sintáctico de descenso recursivo, incluso uno con rastreo hacia atrás, entre en un ciclo infinito. Es decir, al tratar de expandir una no terminal A, podríamos en un momento dado encontrarnos tratando otra vez de expandir a A, sin haber consumido ningún símbolo de la entrada



# PRIMERO y SIGUIENTE

- La construcción de los analizadores sintácticos descendentes y ascendentes es auxiliada por dos funciones, PRIMERO y SIGUIENTE, asociadas con la gramática  $G$ .
    - *Permiten elegir la producción que vamos a aplicar, con base en el siguiente símbolo de entrada.*
    - *Durante la recuperación de errores en modo de pánico, los conjuntos de tokens que produce SIGUIENTE pueden usarse como tokens de sincronización.*
- Definimos a  $\text{PRIMERO}(\alpha)$

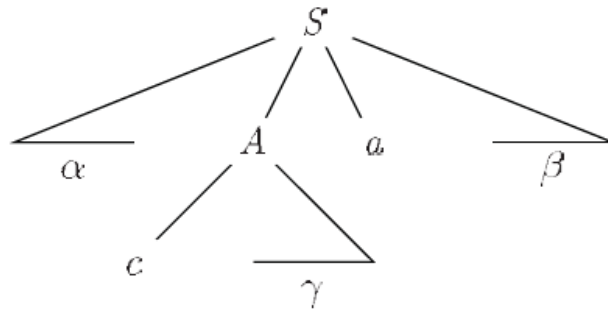


Figura 4.15: El terminal  $c$  está en  $\text{PRIMERO}(A)$  y  $a$  está en  $\text{SIGUIENTE}(A)$

Para calcular  $\text{PRIMERO}(X)$  para todos los símbolos gramaticales  $X$ , aplicamos las siguientes reglas hasta que no pueden agregarse más terminales o  $\epsilon$  a ningún conjunto  $\text{PRIMERO}$ .

1. Si  $X$  es un terminal, entonces  $\text{PRIMERO}(X) = \{X\}$ .
2. Si  $X$  es un no terminal y  $X \rightarrow Y_1 Y_2 \cdots Y_k$  es una producción para cierta  $k \geq 1$ , entonces se coloca  $a$  en  $\text{PRIMERO}(X)$  si para cierta  $i$ ,  $a$  está en  $\text{PRIMERO}(Y_i)$ , y  $\epsilon$  está en todas las funciones  $\text{PRIMERO}(Y_1), \dots, \text{PRIMERO}(Y_{i-1})$ ; es decir,  $Y_1 \cdots Y_{i-1} \xRightarrow{*} \epsilon$ . Si  $\epsilon$  está en  $\text{PRIMERO}(Y_j)$  para todas las  $j = 1, 2, \dots, k$ , entonces se agrega  $\epsilon$  a  $\text{PRIMERO}(X)$ . Por ejemplo, todo lo que hay en  $\text{PRIMERO}(Y_1)$  se encuentra sin duda en  $\text{PRIMERO}(X)$ . Si  $Y_1$  no deriva a  $\epsilon$ , entonces no agregamos nada más a  $\text{PRIMERO}(X)$ , pero si  $Y_1 \xRightarrow{*} \epsilon$ , entonces agregamos  $\text{PRIMERO}(Y_2)$ , y así sucesivamente.
3. Si  $X \rightarrow \epsilon$  es una producción, entonces se agrega  $\epsilon$  a  $\text{PRIMERO}(X)$ .

Para calcular  $\text{SIGUIENTE}(A)$  para todas las no terminales  $A$ , se aplican las siguientes reglas hasta que no pueda agregarse nada a cualquier conjunto  $\text{SIGUIENTE}$ .

1. Colocar  $\$$  en  $\text{SIGUIENTE}(S)$ , en donde  $S$  es el símbolo inicial y  $\$$  es el delimitador derecho de la entrada.
2. Si hay una producción  $A \rightarrow \alpha B \beta$ , entonces todo lo que hay en  $\text{PRIMERO}(\beta)$  excepto  $\epsilon$  está en  $\text{SIGUIENTE}(B)$ .
3. Si hay una producción  $A \rightarrow \alpha B$ , o una producción  $A \rightarrow \alpha B \beta$ , en donde  $\text{PRIMERO}(\beta)$  contiene a  $\epsilon$ , entonces todo lo que hay en  $\text{SIGUIENTE}(A)$  está en  $\text{SIGUIENTE}(B)$ .

# Ejemplo

$$\begin{array}{lll} E & \rightarrow & T E' \\ E' & \rightarrow & + T E' \mid \epsilon \\ T & \rightarrow & F T' \\ T' & \rightarrow & * F T' \mid \epsilon \\ F & \rightarrow & ( E ) \mid \mathbf{id} \end{array} \quad (4.28)$$

**Ejemplo 4.30:** Considere de nuevo la gramática no recursiva por la izquierda (4.28). Entonces:

1.  $\text{PRIMERO}(F) = \text{PRIMERO}(T) = \text{PRIMERO}(E) = \{ (, \mathbf{id} \}$ . Para ver por qué, observe que las dos producciones para  $F$  tienen producciones que empiezan con estos dos símbolos terminales,  $\mathbf{id}$  y el paréntesis izquierdo.  $T$  sólo tiene una producción y empieza con  $F$ . Como  $F$  no deriva a  $\epsilon$ ,  $\text{PRIMERO}(T)$  debe ser igual que  $\text{PRIMERO}(F)$ . El mismo argumento se cumple con  $\text{PRIMERO}(E)$ .
2.  $\text{PRIMERO}(E') = \{ +, \epsilon \}$ . La razón es que una de las dos producciones para  $E'$  tiene un cuerpo que empieza con el terminal  $+$ , y la otra es  $\epsilon$ . Cada vez que un no terminal deriva a  $\epsilon$ , colocamos a  $\epsilon$  en  $\text{PRIMERO}$  para ese no terminal.
3.  $\text{PRIMERO}(T') = \{ *, \epsilon \}$ . El razonamiento es análogo al de  $\text{PRIMERO}(E')$ .

$$\begin{array}{lll}
 E & \rightarrow & T E' \\
 E' & \rightarrow & + T E' \mid \epsilon \\
 T & \rightarrow & F T' \\
 T' & \rightarrow & * F T' \mid \epsilon \\
 F & \rightarrow & ( E ) \mid \mathbf{id}
 \end{array} \tag{4.28}$$

4.  $\text{SIGUIENTE}(E) = \text{SIGUIENTE}(E') = \{ \}, \$\}$ . Como  $E$  es el símbolo inicial,  $\text{SIGUIENTE}(E)$  debe contener  $\$$ . El cuerpo de la producción  $( E )$  explica por qué el paréntesis derecho está en  $\text{SIGUIENTE}(E)$ . Para  $E'$ , observe que esta no terminal sólo aparece en los extremos de los cuerpos de las producciones  $E$ . Por ende,  $\text{SIGUIENTE}(E')$  debe ser el mismo que  $\text{SIGUIENTE}(E)$ .
5.  $\text{SIGUIENTE}(T) = \text{SIGUIENTE}(T') = \{ +, ), \$\}$ . Observe que  $T$  aparece en las producciones sólo seguido por  $E'$ . Por lo tanto, todo lo que esté en  $\text{PRIMERO}(E')$ , excepto  $\epsilon$ , debe estar en  $\text{SIGUIENTE}(T)$ ; eso explica el símbolo  $+$ . No obstante, como  $\text{PRIMERO}(E')$  contiene a  $\epsilon$  (es decir,  $E' \xRightarrow{*} \epsilon$ ), y  $E'$  es la cadena completa que va después de  $T$  en los cuerpos de las producciones  $E$ , todo lo que hay en  $\text{SIGUIENTE}(E)$  también debe estar en  $\text{SIGUIENTE}(T)$ . Eso explica los símbolos  $\$$  y el paréntesis derecho. En cuanto a  $T'$ , como aparece sólo en los extremos de las producciones  $T$ , debe ser que  $\text{SIGUIENTE}(T') = \text{SIGUIENTE}(T)$ .
6.  $\text{SIGUIENTE}(F) = \{ +, *, ), \$\}$ . El razonamiento es análogo al de  $T$  en el punto (5).

$$\begin{array}{lll}
 E & \rightarrow & T E' \\
 E' & \rightarrow & + T E' \mid \epsilon \\
 T & \rightarrow & F T' \\
 T' & \rightarrow & * F T' \mid \epsilon \\
 F & \rightarrow & ( E ) \mid \mathbf{id}
 \end{array} \tag{4.28}$$

# Gramáticas LL(1)

Son analizadores sintácticos predictivos y de descenso recursivo que no necesitan rastreo hacia atrás:

- Pueden construirse para una clase de gramáticas llamadas *LL(1)*.
- La primera “L” en *LL(1)* es para explorar la entrada de izquierda a derecha (por left en inglés).
- La segunda “L” para producir una derivación por la izquierda.
- El “1” para usar un símbolo de entrada de anticipación en cada paso, para tomar las decisiones de acción del análisis sintáctico.

# Características

La clase de gramáticas LL(1) es lo bastante robusta como para cubrir la mayoría de las construcciones de programación, aunque hay que tener cuidado al escribir una gramática adecuada para el lenguaje fuente. Por ejemplo, ninguna gramática recursiva por la izquierda o ambigua puede ser LL(1).

Una gramática  $G$  es LL(1) si, y sólo si cada vez que  $A \rightarrow \alpha \mid \beta$  son dos producciones distintas de  $G$ , se aplican las siguientes condiciones:

1. Para el no terminal  $a$ , tanto  $\alpha$  como  $\beta$  derivan cadenas que empiecen con  $a$ .
2. A lo más, sólo  $\alpha$  o  $\beta$  puede derivar la cadena vacía.
3. Si  $\beta \xRightarrow{*} \epsilon$ , entonces  $\alpha$  no deriva a ninguna cadena que empiece con una terminal en  $\text{SIGUIENTE}(A)$ . De igual forma, si  $\alpha \xRightarrow{*} \epsilon$ , entonces  $\beta$  no deriva a ninguna cadena que empiece con una terminal en  $\text{SIGUIENTE}(A)$ .



# Ejemplo de análisis

Pueden construirse analizadores sintácticos predictivos para las gramáticas LL(1), ya que puede seleccionarse la producción apropiada a aplicar para una no terminal con sólo analizar el símbolo de entrada actual. Los constructores del flujo de control, con sus palabras clave distintivas, por lo general, cumplen con las restricciones de LL(1). Por ejemplo, si tenemos las siguientes producciones:

$$\begin{array}{lcl} instr & \rightarrow & \textbf{if} ( expr ) instr \textbf{else} instr \\ & | & \textbf{while} ( expr ) instr \\ & | & \{ lista\_instr \} \end{array}$$

# Idea de construcción análisis sintácticos predictivos

**ENTRADA:** La gramática  $G$ .

**SALIDA:** La tabla de análisis sintáctico  $M$ .

**MÉTODO:** Para cada producción  $A \rightarrow \alpha$  de la gramática, hacer lo siguiente:

1. Para cada terminal  $a$  en  $\text{PRIMERO}(A)$ , agregar  $A \rightarrow \alpha$  a  $M[A, a]$ .
2. Si  $\epsilon$  está en  $\text{PRIMERO}(\alpha)$ , entonces para cada terminal  $b$  en  $\text{SIGUIENTE}(A)$ , se agrega  $A \rightarrow \alpha$  a  $M[A, b]$ . Si  $\epsilon$  está en  $\text{PRIMERO}(\alpha)$  y  $\$$  se encuentra en  $\text{SIGUIENTE}(A)$ , se agrega  $A \rightarrow \alpha$  a  $M[A, \$]$  también.

# Sea la gramática:

$$\begin{aligned}
 E &\rightarrow T E' \\
 E' &\rightarrow + T E' \mid \epsilon \\
 T &\rightarrow F T' \\
 T' &\rightarrow * F T' \mid \epsilon \\
 F &\rightarrow ( E ) \mid \text{id}
 \end{aligned}$$

- Los espacios en blanco son entradas de error; los espacios que no están en blanco indican una producción con la cual se expande un no terminal.

| No<br>TERMINAL | SÍMBOLO DE ENTRADA        |                           |                         |                       |                           |                           |
|----------------|---------------------------|---------------------------|-------------------------|-----------------------|---------------------------|---------------------------|
|                | id                        | +                         | *                       | (                     | )                         | \$                        |
| $E$            | $E \rightarrow T E'$      |                           |                         | $E \rightarrow T E'$  |                           |                           |
| $E'$           |                           | $E' \rightarrow + T E'$   |                         |                       | $E' \rightarrow \epsilon$ | $E' \rightarrow \epsilon$ |
| $T$            | $T \rightarrow F T'$      |                           |                         | $T \rightarrow F T'$  |                           |                           |
| $T'$           |                           | $T' \rightarrow \epsilon$ | $T' \rightarrow * F T'$ |                       | $T' \rightarrow \epsilon$ | $T' \rightarrow \epsilon$ |
| $F$            | $F \rightarrow \text{id}$ |                           |                         | $F \rightarrow ( E )$ |                           |                           |

Considere la producción  $E \rightarrow TE'$ . Como

$$\text{PRIMERO}(TE') = \text{PRIMERO}(T) = \{ (, \mathbf{id} \}$$

esta producción se agrega a  $M[E, (]$  y  $M[E, \mathbf{id}]$ . La producción  $E' \rightarrow +TE'$  se agrega a  $M[E', +]$ , ya que  $\text{PRIMERO}(+TE') = \{+\}$ . Como  $\text{SIGUIENTE}(E') = \{ ), \$ \}$ , la producción  $E' \rightarrow \epsilon$  se agrega a  $M[E', )]$  y a  $M[E', \$]$ .  $\square$

$$\text{PRIMERO}(F) = \text{PRIMERO}(T) = \text{PRIMERO}(E) = \{ (, \mathbf{id} \}.$$

$$\text{PRIMERO}(E') = \{ +, \epsilon \}.$$

$$\text{PRIMERO}(T') = \{ *, \epsilon \}.$$

$$\text{SIGUIENTE}(E) = \text{SIGUIENTE}(E') = \{ ), \$ \}.$$

$$\text{SIGUIENTE}(T) = \text{SIGUIENTE}(T') = \{ +, ), \$ \}.$$

$$\text{SIGUIENTE}(F) = \{ +, *, ), \$ \}.$$

$$\begin{array}{lll} E & \rightarrow & T E' \\ E' & \rightarrow & + T E' \mid \epsilon \\ T & \rightarrow & F T' \\ T' & \rightarrow & * F T' \mid \epsilon \\ F & \rightarrow & ( E ) \mid \mathbf{id} \end{array}$$

# Estudiar:

- El ejemplo 4.33
- La sección 4.4.4 Análisis sintáctico predictivo no recursivo

# Taller en clase:

1. Para cada una de las siguientes gramáticas. Hallar los conjuntos PRIMEROS (FIRST) y SIGUIENTE (FOLLOW):

a.  $S \rightarrow 0S1 \mid 01$

b.  $S \rightarrow + S S \mid * S S \mid a$

c.  $S \rightarrow S ( S ) S \mid \varepsilon$

d.  $S \rightarrow S + S \mid S S \mid ( S ) \mid S * \mid a$

e.  $S \rightarrow ( L ) \mid a$  y  $L \rightarrow L, S \mid S$

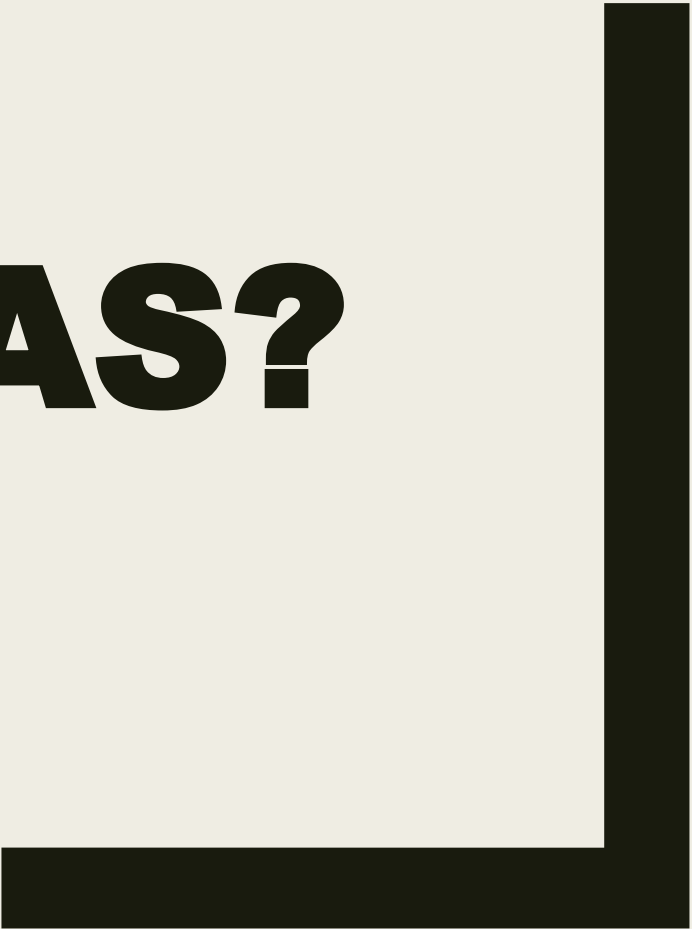
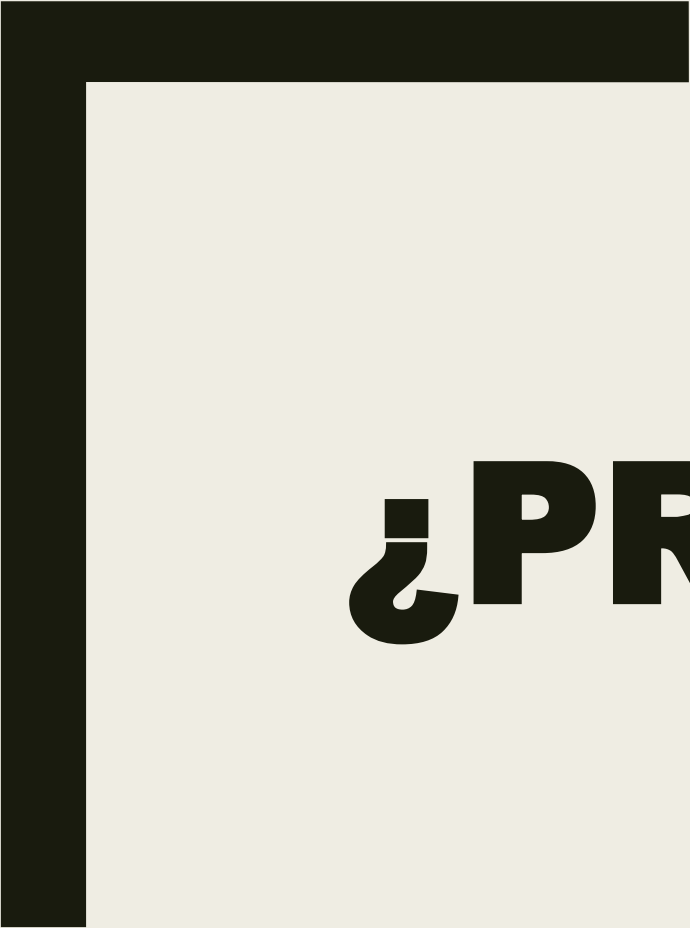
f.  $S \rightarrow aSbS \mid bSaS \mid \varepsilon$

g.  $BEXPR \rightarrow BEXPR \text{ or } BTERM \mid BTERM$

$BTERM \rightarrow BTERM \text{ and } BFACTOR \mid BFACTOR$

$BFACTOR \rightarrow \text{not } BFACTOR \mid (BEXPR) \mid \text{true} \mid \text{false}$

2. Para cada una de las gramáticas del punto 1, idee analizadores sintácticos predictivos y muestre las tablas de análisis sintáctico. Puede factorizar por la izquierda o eliminar la recursividad por la izquierda de sus gramáticas primero si aplica.



**¿PREGUNTAS?**